Computer Systems Architecture
Homework 2

Ashesh Kaji

Due: February 22

Problem 1.

Solution.

(a) Since the machine uses 4-byte words, `A[10]` is at $4 \times 10 = 40$ bytes from the base address of A (`x27`).

```
lw x5, 40(x27)
```

(b) Similar to the previous load, storing into `A[17]` dictates a byte offset of $17 \times 4 = 68$ from base `x27`.

```
sw x5, 68(x27)
```

(c) `add x7, x5, x6`

(d) This operation breaks down into computing the source address, loading the value, computing the destination address, and storing the value:

```
add  x8, x28, x29 # calculating i + j
addi x8, x8, 31 # calculating i + j + 31
slli x8, x8, 2 # byte offset for A[i + j + 31]
add  x8, x27, x8 # full address of A[i + j + 31]
lw   x8, 0(x8) # loading A[i + j + 31] into x8
slli x9, x6, 2 # byte offset for C[g] (g * 4)
add  x9, x31, x9 # full address of C[g]
sw   x8, 0(x9) # storing the loaded value into C[g]
```

(e)

(i)

```
lw   x8, 36(x30) # loading B[9] (9 * 4 = 36)
slli x8, x8, 2 # byte offset for A[B[9]]
add  x8, x27, x8 # full address of A[B[9]]
lw   x8, 0(x8) # loading A[B[9]]
sub  x5, x6, x8 # f = g - A[B[9]]
```

(ii)

```
lw   x8, 32(x31) # loading C[8] (8 * 4 = 32)
lw   x9, 16(x30) # loading B[4] (4 * 4 = 16)
add  x8, x8, x9 # C[8] + B[4]
slli x8, x8, 2 # byte offset for A index
add  x8, x27, x8 # full address of A[C[8] + B[4]]
lw   x8, 0(x8) # loading A[C[8] + B[4]]
sub  x5, x6, x8 # f = g - A[C[8] + B[4]]
```

(iii)

```
slli x8, x28, 3 # byte offset for B[2i] (2i * 4 = 8i)
add  x9, x30, x8 # full address of B[2i]
lw   x10, 0(x9) # loading B[2i]
slli x11, x28, 2 # byte offset for C[i] and A[i]
add  x12, x31, x11 # full address of C[i]
sw   x10, 0(x12) # C[i] = B[2i]
addi x9, x9, 4 # full address of B[2i + 1] (B[2i] + 4)
lw   x10, 0(x9) # loading B[2i + 1]
add  x12, x27, x11 # full address of A[i]
sw   x10, 0(x12) # A[i] = B[2i + 1]
```

(iv) `A[i] = 4*B[i-1] + 4*C[i+1]`

```
slli x8, x28, 2 # byte offset for i
addi x9, x8, -4 # byte offset for i-1
add  x9, x30, x9 # address of B[i-1]
lw   x9, 0(x9) # loading B[i-1]
addi x10, x8, 4 # byte offset for i+1
add  x10, x31, x10 # address of C[i+1]
lw   x10, 0(x10) # loading C[i+1]
slli x9, x9, 2 # multiplying B[i-1] by 4
slli x10, x10, 2 # multiplying C[i+1] by 4
add  x9, x9, x10 # adding 4*B[i-1] + 4*C[i+1]
add  x8, x27, x8 # address of A[i]
sw   x9, 0(x8) # storing result into A[i]
```

(v) `f = g - A[C[4] + B[12]]`

```
lw   x8, 16(x31) # loading C[4]
lw   x9, 48(x30) # loading B[12]
add  x8, x8, x9 # calculating C[4] + B[12]
slli x8, x8, 2 # multiplying by 4 for byte offset
add  x8, x27, x8 # address of A[C[4]+B[12]]
lw   x8, 0(x8) # loading A[C[4]+B[12]]
sub  x5, x6, x8 # subtracting from g
```

Problem 2.

Given: `x5 = 0x00000000AAAAAAAA`, `x6 = 0x1234567812345678`

Solution.

(a) Tracing each instruction step by step:

- `srli x7, x5, 16` — logical right shift x5 by 16 bits (zero-fill):

$$\texttt{x7 = 0x000000000000AAAA}$$

- `addi x7, x7, -128` — add -128 (0xFFFFFFFFFFFFFF80) to x7:

$$\texttt{0x000000000000AAAA} + \texttt{(-128)} = \texttt{0x000000000000AA2A}$$

- `srai x7, x7, 2` — arithmetic right shift by 2 (sign bit is 0, so same as logical):

$$\texttt{0x000000000000AA2A} \gg 2 = \texttt{0x0000000000002A8A}$$

- `and x7, x7, x6` — bitwise AND with x6:

$$\texttt{0x0000000000002A8A} \,\&\, \texttt{0x1234567812345678} = \texttt{0x0000000000000208}$$

Result: x7 = `0x0000000000000208`

(b) Tracing the instruction:

- `slli x7, x6, 4` — logical left shift x6 by 4 bits. The top nibble `0x1` is shifted out:

$$\texttt{0x1234567812345678} \ll 4 = \texttt{0x2345678123456780}$$

Result: x7 = `0x2345678123456780`

(c) Tracing each instruction step by step:

- `srli x7, x5, 3` — logical right shift x5 by 3 bits:

$$\texttt{0x00000000AAAAAAAA} \gg 3 = \texttt{0x0000000015555555}$$

- `andi x7, x7, 0xFEF` — `andi` takes a 12-bit signed immediate. `0xFEF` has its sign bit (bit 11) set, so it sign-extends to `0xFFFFFFFFFFFFFEF`:

$$\texttt{0x0000000015555555} \,\&\, \texttt{0xFFFFFFFFFFFFFEF} = \texttt{0x0000000015555545}$$

Result: x7 = `0x0000000015555545`

Problem 3.

Solution.
Notes on RISC-V formats used:

- **R-type**: `funct7 | rs2 | rs1 | funct3 | rd | opcode`
- **I-type**: `imm[11:0] | rs1 | funct3 | rd | opcode`
- **S-type**: `imm[11:5] | rs2 | rs1 | funct3 | imm[4:0] | opcode`
- **B-type**: `imm[12|10:5] | rs2 | rs1 | funct3 | imm[4:1|11] | opcode`
- **U-type**: `imm[31:12] | rd | opcode`
- **J-type**: `imm[20|10:1|11|19:12] | rd | opcode`

Instruction	Format	opcode	rs1	rs2	rd	imm	funct3	funct7	32-bit hex
`add x5, x6, x7`	R	0110011	x6 (6)	x7 (7)	x5 (5)	—	000	0000000	0x007302B3
`addi x8, x5, 512`	I	0010011	x5 (5)	—	x8 (8)	512 (0x200)	000	—	0x20028413
`ld x3, 128(x27)`	I	0000011	x27 (27)	—	x3 (3)	128 (0x080)	011	—	0x080DB183
`sd x3, 256(x28)`	S	0100011	x28 (28)	x3 (3)	—	256 (0x100)	011	—	0x103E3023
`beq x5, x6, ELSE`	B	1100011	x5 (5)	x6 (6)	—	+16	000	—	0x00628863
`add x3, x0, x0`	R	0110011	x0 (0)	x0 (0)	x3 (3)	—	000	0000000	0x000001B3
`auipc x3, 0xFFEFA`	U	0010111	—	—	x3 (3)	0xFFEFA	—	—	0xFFEFA197
`jal x3, ELSE`	J	1101111	—	—	x3 (3)	+16	—	—	0x010001EF

Notes:

- `beq` and `jal`: ELSE is 16 bytes ahead of the current PC, so the branch/jump offset is +16.

- `sd x3, 256(x28)`: the 12-bit immediate 256 is split as `imm[11:5] = 0001000` and `imm[4:0] = 00000`.
- `auipc x3, 0xFFEFA`: places `0xFFEFA000 + PC` into x3; the 20-bit immediate occupies bits [31:12].
- `add x3, x0, x0`: effectively zeroes x3 (a common RISC-V idiom).

Problem 4.

Solution.

(a) Assuming `x5` holds variable A and `x11` holds the base address of C:

```
lw   x5, 0(x11) # loading C[0] into x5
slli x5, x5, 16 # A = C[0] << 16
```

(b) Using a 5-instruction XOR swap trick to replace the bits without needing multiple shift-and-mask operations:

```
slli x6, x3, 16 # shift bits 12:7 of x3 to position 28:23
xor  x6, x4, x6 # x6 = difference between x4 and new masked bits
lui  x7, 0x1F800 # x7 = 0x1F800000 (mask for bits 28:23)
and  x6, x6, x7 # keep only the difference inside the mask
xor  x4, x4, x6 # apply changes to x4, leaving other bits untouched
```

Testing output: If `x3 = 0` and `x4 = 0xffffffffffffffff`, then `x6` becomes `0^x4 = 0xff...ff`, masked to `0x1F800000`, and explicitly flipping these bits via XOR makes bits 28:23 in `x4` equal to 0 exactly where configured, yielding `0xffffffffe07fffff`.

(c) Since there is no direct `not` instruction in base RISC-V, we use XOR with the all-ones constant (-1 fits implicitly in the 12-bit signed immediate):

```
xori x5, x6, -1 # bit-wise invert of x6, stored into x5
```

Problem 5.

Solution.

(a) The `jal` instruction utilizes the J-type format, encoding a 20-bit immediate. This immediate is left-shifted by 1 bit, resulting in a 21-bit signed byte offset relative to the PC. The reachable offset range is $[-2^{20}, 2^{20}-2]$ bytes, or $[-0\text{x}100000, +0\text{x}0\text{FFFFE}]$. Given PC = `0x60000000`, the range of addresses that can be reached is:

$$[\texttt{0x5FF00000}, \texttt{0x600FFFFE}]$$

(b) The `beq` instruction utilizes the B-type format, encoding a 12-bit immediate. This immediate is left-shifted by 1 bit, resulting in a 13-bit signed byte offset relative to the PC. The reachable offset range is $[-2^{12}, 2^{12}-2]$ bytes, or $[-0\text{x}1000, +0\text{x}0\text{FFE}]$. Given PC = `0x60000000`, the range of addresses that can be reached is:

$$[\texttt{0x5FFFF000}, \texttt{0x60000FFE}]$$

Problem 6. Based on the provided assembly, the initialization state is `x6 = 10` and `x5 = 0`. The loop decrements `x6` by 1 and increments `x5` by 2 per iteration, terminating when `x6 == 0`. Thus, the loop executes 10 times, leaving the final value of `x5` as 20.

Solution.

(a) Assigning `acc` to `x5` and `i` to `x6`, the equivalent C code is a conditional loop that subtracts from `i` and adds to `acc` until `i` reaches zero:

```c
while (i != 0) {
    i--;
    acc += 2;
}
```

(b) Assuming $N \geq 0$, the structure executes 4 instructions per iteration ('`beq`', '`addi`', '`addi`', '`jal`'). Upon the final evaluation when `x6 == 0`, only the '`beq`' instruction executes before the loop terminates. Thus, the total number of executed instructions is mathematically modeled as:

$$4N + 1$$

(c) Replacing the branch instruction with `blt x6, x0, DONE` modifies the exit condition to break only when `x6 < 0`. The equivalent C logic translates to checking if `i >= 0`:

```c
while (i >= 0) {
    i--;
    acc += 2;
}
```

Problem 7.

Solution.

(a) Utilizing a standard top-tested `for` loop structure to structurally match the C code. In memory, each sequence element in `D` is a 4-byte architectural integer, making the logical index expression `4*j` evaluate to a byte offset of $4 \times 4 \times j = 16j$. We formulate this scaled offset via a logical left shift by 4 (`slli`):

```r
    add  x7, x0, x0 # i = 0
OUTER_LOOP:
    bge  x7, x5, DONE # exit outer loop if (i >= a)
    add  x29, x0, x0 # j = 0
INNER_LOOP:
    bge  x29, x6, INNER_DONE # exit inner loop if (j >= b)
    add  x11, x7, x29 # x11 = i + j
    slli x12, x29, 4 # x12 = 16 * j (byte offset for index 4j)
    add  x12, x10, x12 # x12 = array base address + offset
    sw   x11, 0(x12) # D[4*j] = i + j
    addi x29, x29, 1 # j++
    jal  x0, INNER_LOOP # loop iteration vector
INNER_DONE:
    addi x7, x7, 1 # i++
    jal  x0, OUTER_LOOP # loop iteration vector
DONE:
```

(b) The complete implementation translates into **12 assembly instructions**. Assuming initialization inputs bounded at logical extrema $a = 10$ and $b = 1$, the total program execution spans:

- **Outer initialization**: The structural `add` loop initializer executes 1 time.
- **Outer conditional**: The testing bounds limit evaluates 11 times.
- **Inner initialization**: The inner assignment logic sets up exactly 10 times.
- **Inner conditional**: Testing inner limits bounds evaluates 2 times ($j = 0$ and $j = 1$) during every i cycle, totaling 20.

- **Inner logic payload**: The 6 sequence instructions loop execute logically for iteration index space ($10 \times 1 = 10$). Hence, 60 execution increments.
- **Outer step function**: The exit bounds index modifier runs dynamically following each inner loop termination exactly 10 times. Total bounds: 20 sequential instructions executed.

The sum compiles analytically as mathematically complete sequences:

$$1 + 11 + 10 + 20 + 60 + 20 = 122 \text{ executed instructions.}$$

Problem 8. The sequence utilizes `lb` to load an 8-bit byte symmetrically mapped from `0x10000000` alongside `sd` which overwrites the memory frame extending boundaries `0x10000008`–`0x1000000F`. Thus, the memory value precisely at address `0x10000007` remains unmutated from its initial mapping.
Solution.
(a) In a **big-endian** mapping model, the most significant byte of the 64-bit construct `0x1122334455667788` aligns dynamically to the lowest address `0x10000000`. Consequently, the least significant byte maps symmetrically to `0x10000007`.

Value at 0x10000007: `0x88`

(b) In a **little-endian** mapping model, the lowest address stores the least significant memory slice, dictating the ascending array order. Resulting symmetrically, the most significant byte establishes itself at exactly the highest address `0x10000007`.

Value at 0x10000007: `0x11`

Problem 9.
Solution. Formulating strictly the 64-bit exact identical symmetric sequence `0x1234567812345678` analytically utilizes modular shifts to construct 32-bit components, replicating via logical add structures inside registers:

```
lui  x10, 0x12345 # load upper immediate: x10 = 0x12345000
addi x10, x10, 0x678 # sign-ext 0x678 adds perfectly: x10 = 0x12345678
slli x11, x10, 32 # shift entire sequence to high order bounds
add  x10, x10, x11 # compose duplicated pattern into x10 structurally
```

Problem 10. Given `x5 = 128` functionally initialized, we map limits across the 32-bit bound domains algebraically: $MAX = 2147483647$, $MIN = -2147483648$.
Solution.
(a) For `add x30, x5, x6`, overflow occurs identically when $128 + x6 > 2147483647$. Solving logically: $x6 > 2147483519$. No negative overflow bounds limit this.

Range: $[2147483520,\ 2147483647]$

(b) For `sub x30, x5, x6`, overflow models $128 - x6 > 2147483647 \implies x6 < -2147483519$. Only the minimum signed limit functionally maps an exit bounds domain.

Range: $[-2147483648,\ -2147483520]$

(c) For `sub x30, x6, x5`, overflow structures mathematically across $x6 - 128 < -2147483648$, resolving into a precise sub-bound exit limit.

Range: $[-2147483648,\ -2147483521]$

Problem 10 (Reading Analysis).

Solution.

- **1.1**: Technology scaling decreases active transistor die area, lowering manufacturing costs per transistor. According to Mark Horowitz (ISSCC 2014 keynote), this exponential reduction in the cost of computing enables the widespread global computation and proliferation of computing devices.
- **1.2**: Power density functionally throttled clock frequencies since thermal power heat limits (power dissipation) maximized viable cooling architectures. Since wire resistance scales poorly, pushing higher frequencies causes unacceptable power consumption making systems heavily energy-limited (Horowitz, 2014).
- **1.3**: Moore's Law slows due to lithography costs and atomic-scale physics limits. Concurrently, Dennard Scaling ended since threshold voltages and leakage currents could no longer scale proportionally with physical dimensions (Horowitz, 2014).
- **1.4**: Moving data, particularly off-chip memory access, dominates system energy. Horowitz highlights that external memory bound reads map immense proportional energy loads—scaling orders of magnitude higher than on-die logic ALUs computations.
- **1.5**: Horowitz envisions improvements via hardware-software co-design explicitly targeting heterogeneous computing structures, minimizing off-chip memory calls, and utilizing analytical domain-specific architectures to optimize energy metrics efficiently.

Problem 11.

Solution.

- **(1) Proprietary vs Open-Source ISA Debate:** Reviewing the 2015 IEEE Micro debate (Hill et al., "Proprietary versus Open Instruction Sets"), the tension structures between innovation flexibility versus commercial reliability. Patterson argues an open-source ISA (like RISC-V) unlocks specialized logic dynamically without arbitrary licensing bounds, avoids vendor lock-in, and promotes a unified community ecosystem ensuring stability. Conversely, Christie notes that proprietary ISAs offer a singular, highly tested, compliant ecosystem, mitigating fragmentation and heavy validation testing overheads. Open-source execution responds by crowd-sourcing these validation burdens community-wide.
- **(2) Technical Merits of Open-Source ISAs:** According to Asanović et al. ("Instruction Sets Should Be Free: The Case for RISC-V", UCB EECS-2014-146), RISC-V is unburdened by retroactive compatibility models that weigh down proprietary systems like x86. This pure execution structure yields smaller architectural footprints. Furthermore, customized extension domains organically lower operational constraints, scaling power and geometric area to fit targeted functional domains cleanly.